Movie Recommendation System Using Content-Based Filtering and Cosine Similarity

(Movie Recommendation System Using Machine Learning)

Problem Statement:

With the explosive expansion of digital entertainment and streaming platforms, users face severe cognitive information overload when attempting to select movies aligned with their narrative preferences. Traditional collaborative filtering methods rely on user rating matrices, which severely limits their utility when encountering new or niche movies (the classic "cold-start" challenge). Furthermore, user rating patterns are often sparse, noisy, and subject to external demographic biases.

There is, therefore, a need for an automated, fast, and accessible system that can analyze a movie's inherent metadata (plot description, genres, thematic keywords, cast, and director) and recommend highly similar films with a measurable mathematical confidence score. This project designs and implements a web-based Movie Recommendation System that performs data preprocessing, TF-IDF feature extraction, and similarity computation using Cosine Similarity on the benchmark TMDB 5000 Movie Dataset, and evaluates the recommendation quality using standard content-based metrics (Genre Overlap Ratio, Jaccard Similarity, and Baseline Comparison).

Objectives:

1. To collect and preprocess a comprehensive dataset of movie metadata (TMDB 5000 dataset).
2. To parse complex JSON columns into clean textual features (overview, genres, keywords, cast, director).
3. To construct a domain-weighted combined_features textual representation for each movie.
4. To perform Exploratory Data Analysis (EDA) on genre distributions, release years, vote ratings, and popularity.
5. To extract a 10,000-dimensional unigram and bigram numerical feature vector using Scikit-learn's TF-IDF Vectorizer.
6. To compute on-demand Cosine Similarity scores to rank and retrieve candidate recommendations.
7. To evaluate the recommendation model on a sample query set and report Genre Overlap Ratio and Baseline Gain.
8. To build a Flask-based backend and web front-end that lets a user select a movie and view real-time recommendations.

Software / Hardware Required:

- Operating System: Windows / Linux / macOS
- IDE: Visual Studio Code
- Language: Python 3.12.x
- Backend Framework: Flask (REST API & Local Web Server)

- Machine Learning: scikit-learn (TfidfVectorizer, cosine_similarity)
- Data Processing: Pandas, NumPy
- Data Visualization: Matplotlib, Seaborn
- Model Serialization: Joblib
- Frontend: HTML5, CSS3, JavaScript (Cinematic Dark UI)
- Dataset: TMDB 5000 Movie Dataset (4,803 raw titles; 4,800 usable cleaned records)
- Version Control: Git

Project Directory Structure:

The project ("MovieRecommendationSystem") is organized as a Flask backend with a separate dataset, trained-model, and frontend static/templates folder, as shown below.



Fig 1: Project directory structure of MovieRecommendationSystem in Visual Studio Code

Procedure:

1. Collect the TMDB 5000 movie dataset (movies and credits CSVs) and place them in the data/ folder.
2. Run train_model.py to clean, parse, and organize the raw dataset into the project's processed structure.
3. Preprocess metadata — extract director, cast, keywords, and genres from JSON; handle missing plot overviews.
4. Construct a weighted combined_features textual column with duplicated keywords, genres, and director tokens.
5. Generate exploratory visualizations (genre frequencies, release year histogram, popularity, ratings) into static/plots/.
6. Extract a 10,000-dimensional TF-IDF feature space from the combined text using unigrams and bigrams with English stop-words.
7. Calculate Cosine Similarity angles between document vectors to quantify narrative proximity bounded in [0, 1].
8. Evaluate the similarity engine across 50 query movies (500 recommendations) and report Genre Overlap Ratio and Jaccard alignment.
9. Save the fitted vectorizer, sparse matrix, and lookup tables to models/movie_recommender.pkl using Joblib.
10. Launch the Flask web server (app.py) and query recommendations dynamically via the interactive browser interface.

Implementation Code:

Representative implementation of the key modules used in the project, based on the pipeline shown in the application (adapt variable/file names to match your exact source files).

1. Data Preprocessing & JSON Parsing (train_model.py)

```
import ast, pandas as pd

def safe_parse_json_list(text, key='name', limit=None):
    if not isinstance(text, str) or not text.strip():
        return []
    try:
        data = ast.literal_eval(text)
        if isinstance(data, list):
            items = [item[key] for item in data if isinstance(item, dict) and key in item]
            return items[:limit] if limit else items
    except Exception:
        pass
    return []

def extract_director(crew_text):
    try:
        data = ast.literal_eval(crew_text)
        for member in data:
            if member.get('job') == 'Director':
                return member.get('name', '').strip()
    except Exception:
        pass
    return ''
```

2. Feature Extraction & Combined Features (train_model.py)

```
# Retain both natural names (for bigrams) and unspaced tokens
merged_df['clean_overview'] = merged_df['overview'].apply(clean_text)
merged_df['clean_genres'] = merged_df['genres_text'].apply(clean_text)
merged_df['clean_keywords'] = merged_df['keywords_text'].apply(clean_text)
merged_df['clean_cast'] = merged_df['cast_list'].apply(
    lambda x: ' '.join([f"{clean_text(n)} {clean_text(n).replace(' ', '')}" for n in x])
)
merged_df['clean_director'] = merged_df['director'].apply(
    lambda x: f"{clean_text(x)} {clean_text(x).replace(' ', '')} {clean_text(x).replace(' ', '')}" i
f x else ''
)

# Domain-weighted combined features
merged_df['combined_features'] = (
    merged_df['clean_overview'] + ' ' +
    merged_df['clean_genres'] + ' ' + merged_df['clean_genres'] + ' ' +
    merged_df['clean_keywords'] + ' ' + merged_df['clean_keywords'] + ' ' +
    merged_df['clean_cast'] + ' ' +
    merged_df['clean_director']
).str.strip()
```

3. Model Training & Serialization (train_model.py)

```
from sklearn.feature_extraction.text import TfidfVectorizer
import joblib

# 10,000-dimensional unigram/bigram TF-IDF vectorizer
tfidf = TfidfVectorizer(
    stop_words='english',
    ngram_range=(1, 2),
    max_features=10000,
    sublinear_tf=True
)

# Fit and transform combined text
tfidf_matrix = tfidf.fit_transform(processed_df['combined_features'])
print('TF-IDF Matrix Shape:', tfidf_matrix.shape)

# Persist recommendation model artifacts
artifacts = {
    'tfidf_vectorizer': tfidf,
    'tfidf_matrix': tfidf_matrix,
    'movies_df': processed_df[['id', 'title', 'genres_text', 'overview', 'release_year', 'popularity
']],
    'evaluation_metrics': eval_metrics
}
joblib.dump(artifacts, 'models/movie_recommender.pkl')
```

4. Prediction & Recommendation Route (app.py)

```
from flask import Flask, request, jsonify, render_template
from sklearn.metrics.pairwise import cosine_similarity
import joblib, numpy as np

app = Flask(__name__)
artifacts = joblib.load('models/movie_recommender.pkl')
MOVIES_DF = artifacts['movies_df']
TFIDF_MATRIX = artifacts['tfidf_matrix']

def get_recommendations(movie_idx, genre_filter='all', top_n=5):
    query_vector = TFIDF_MATRIX[movie_idx]
    sim_scores = cosine_similarity(query_vector, TFIDF_MATRIX).flatten()
    sim_scores[movie_idx] = -1.0 # Exclude query movie

    if genre_filter and genre_filter.lower() != 'all':
        mask = MOVIES_DF['genres_text'].str.contains(genre_filter, case=False, na=False).values
        sim_scores = np.where(mask, sim_scores, -1.0)

    ranked_indices = np.argsort(sim_scores)[::-1][:top_n]
    recommendations = []
    for idx in ranked_indices:
        row = MOVIES_DF.iloc[idx]
        recommendations.append({
            'title': row['title'],
            'release_year': row['release_year'],
            'genres': row['genres_text'],
            'similarity_score': round(float(sim_scores[idx]), 4),
            'similarity_pct': round(float(sim_scores[idx]) * 100, 1)
        })
    return recommendations

@app.route('/recommend', methods=['POST'])
def recommend():
    movie_title = request.form.get('movie_title', '').strip()
    genre_filter = request.form.get('genre_filter', 'all').strip()
    top_n = int(request.form.get('num_recommendations', 5))

    movie_idx, suggestions = find_movie_by_title(movie_title)
    if movie_idx is None:
        return render_template('index.html', error_message='Movie not found', suggestions=suggestion
s)

    recs = get_recommendations(movie_idx, genre_filter=genre_filter, top_n=top_n)
    return render_template('index.html', selected_movie=MOVIES_DF.iloc[movie_idx], recommendations=r
ecs)
```

5. Client Autocomplete API (app.py)

```
@app.route('/api/movies', methods=['GET'])
def api_movies():
    all_titles = sorted(MOVIES_DF['title'].dropna().unique().tolist())
    all_genres = sorted(list(set(g for text in MOVIES_DF['genres_text'] for g in text.split())))
    return jsonify({'titles': all_titles, 'genres': all_genres})
```

Output (Step-by-Step Screenshots):

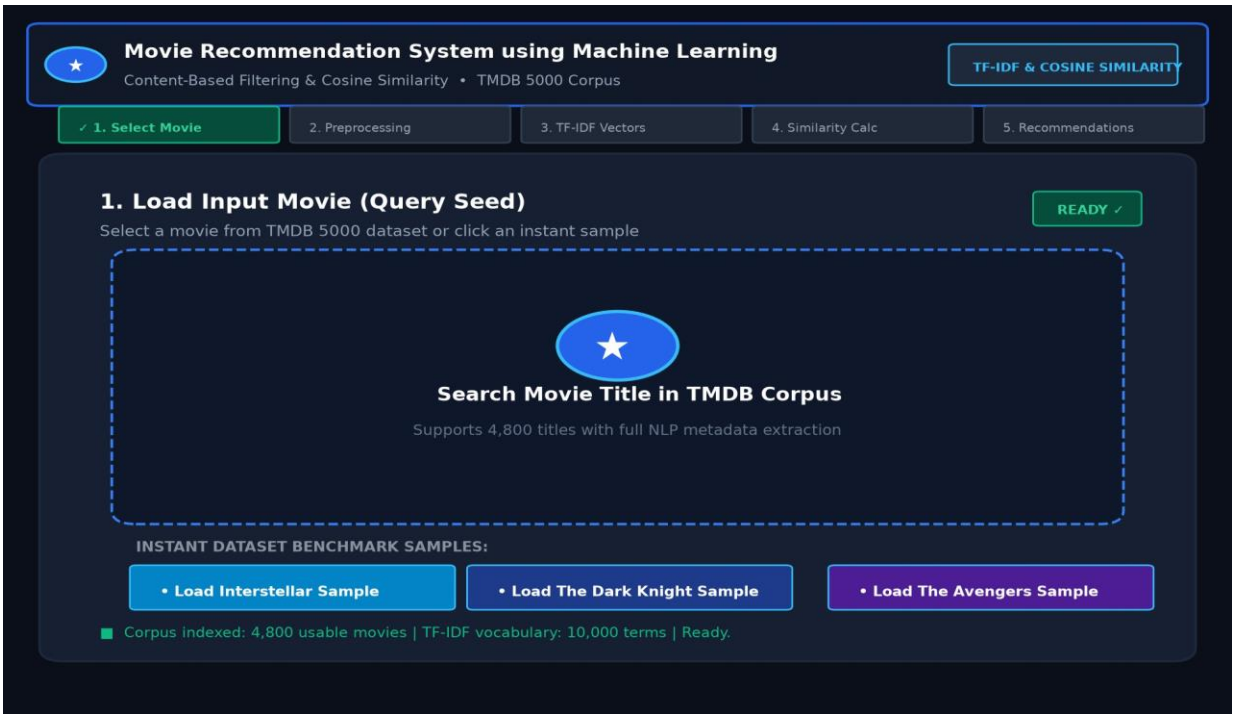


Fig 2: Step 1 — Load Input Movie: interface to select a movie or load a benchmark sample

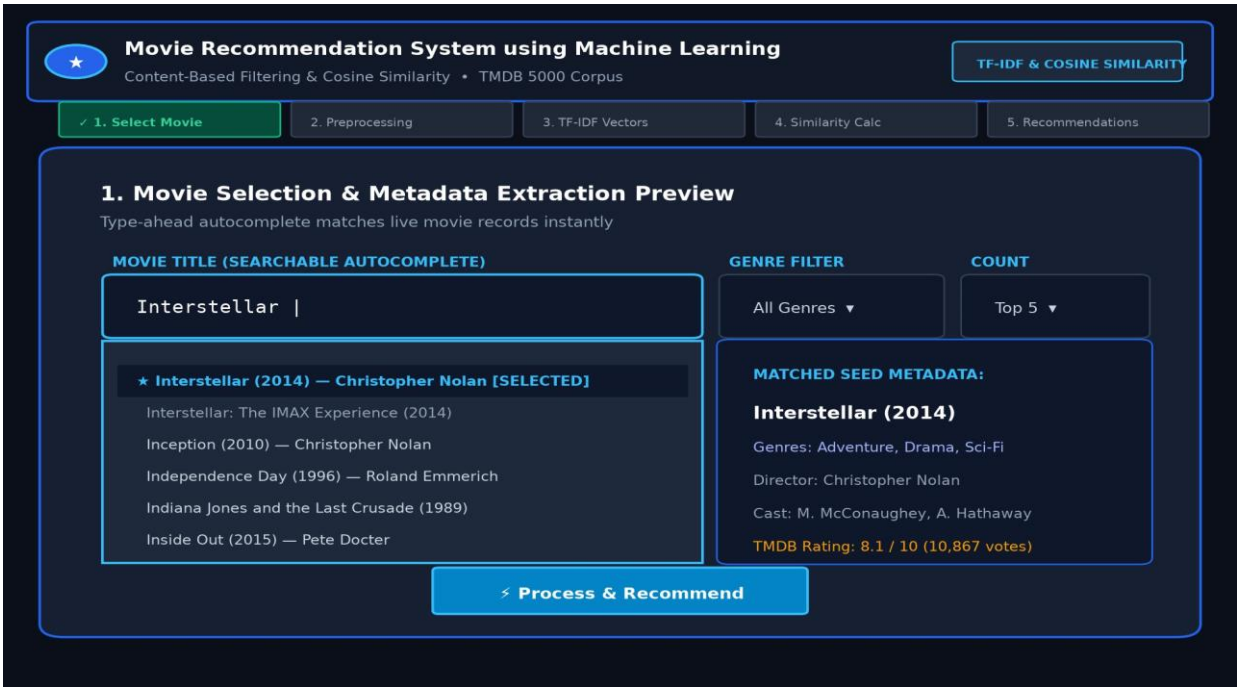


Fig 3: Step 1 — Movie selection preview with searchable autocomplete and instant sample loader

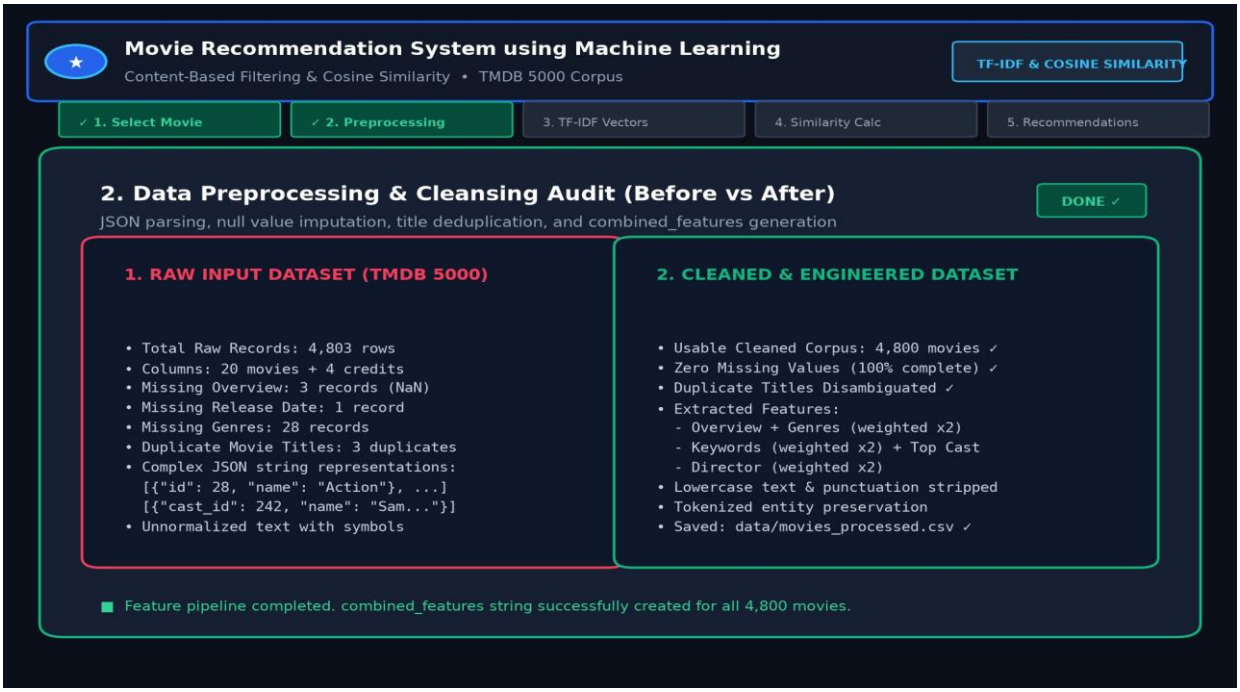


Fig 4: Step 2 — Data preprocessing: raw TMDB dataset vs. cleaned & normalized (4,800 records)

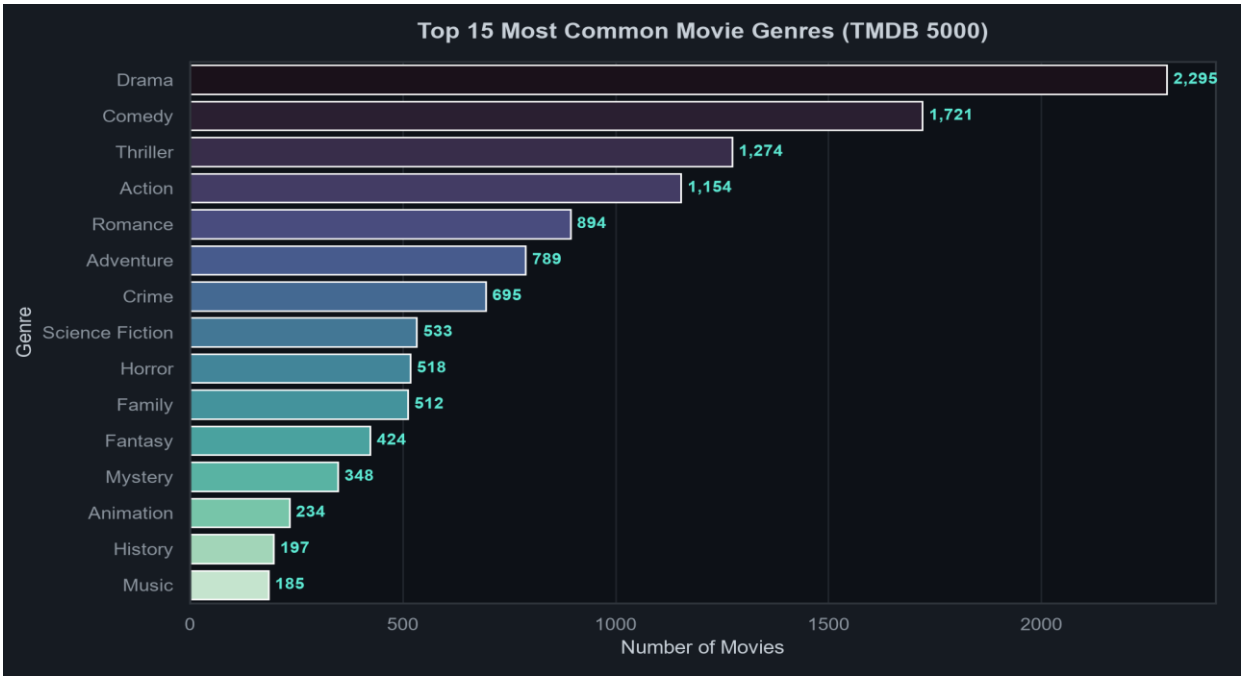


Fig 5: Step 3 — Exploratory Data Analysis: frequency distribution of top 15 movie genres in TMDB 5000

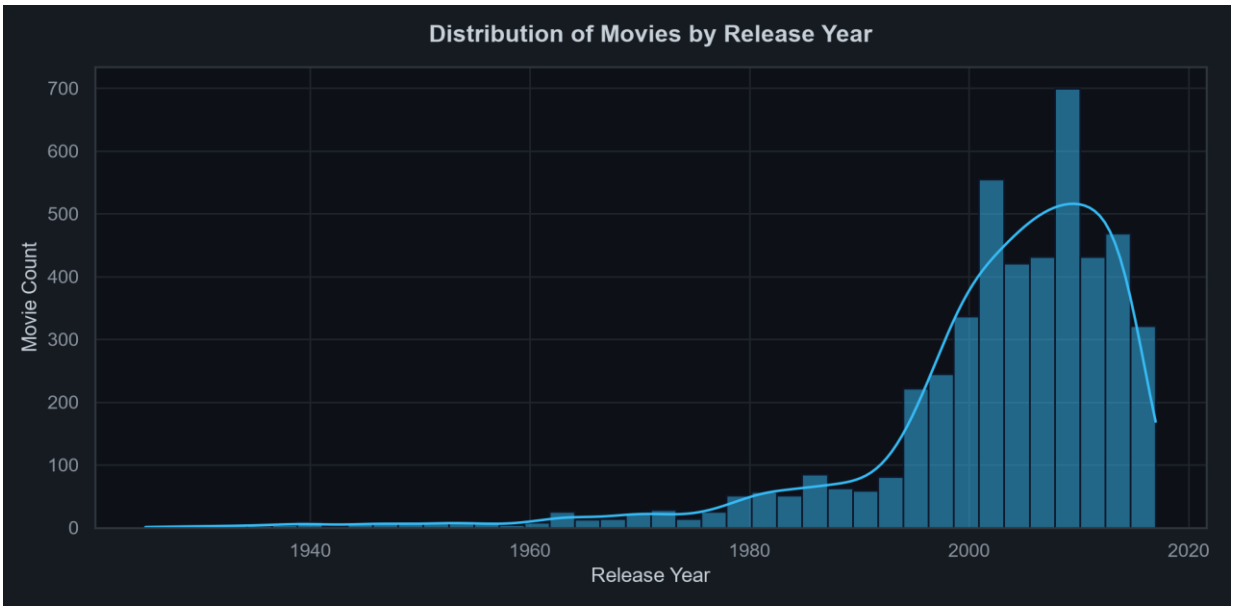


Fig 6: Step 3 — Exploratory Data Analysis: distribution of movies by release year (1920 – 2016)

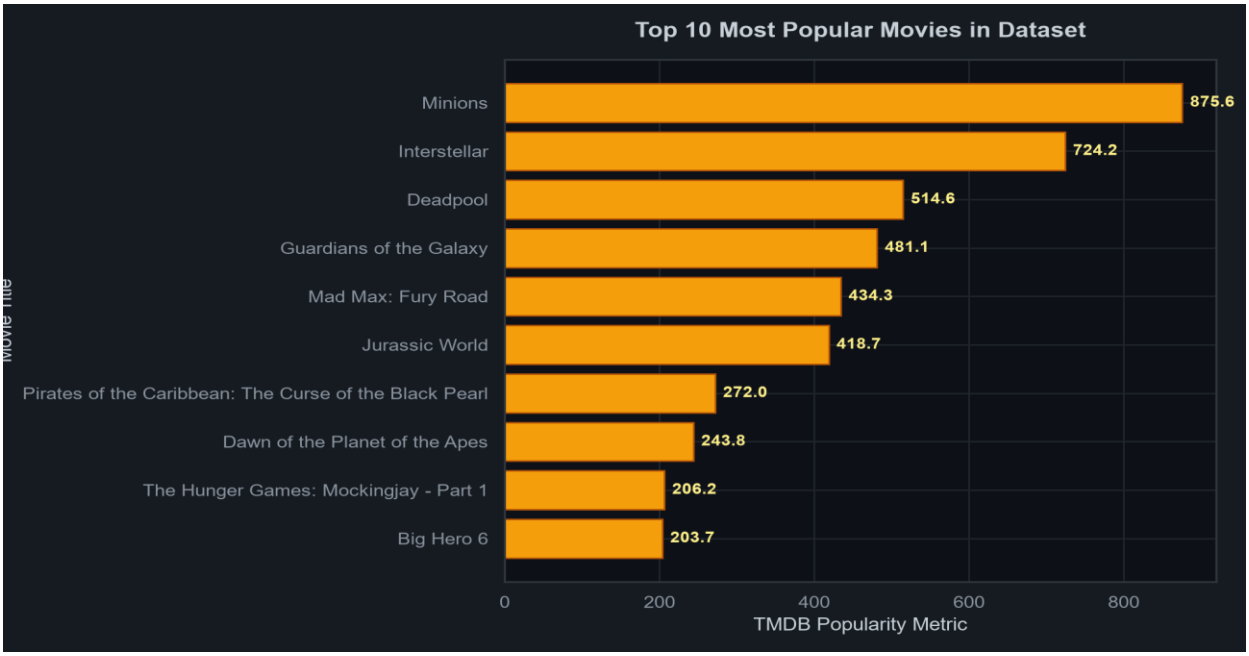


Fig 7: Step 4 — Exploratory Data Analysis: top 10 most popular movies in TMDB dataset

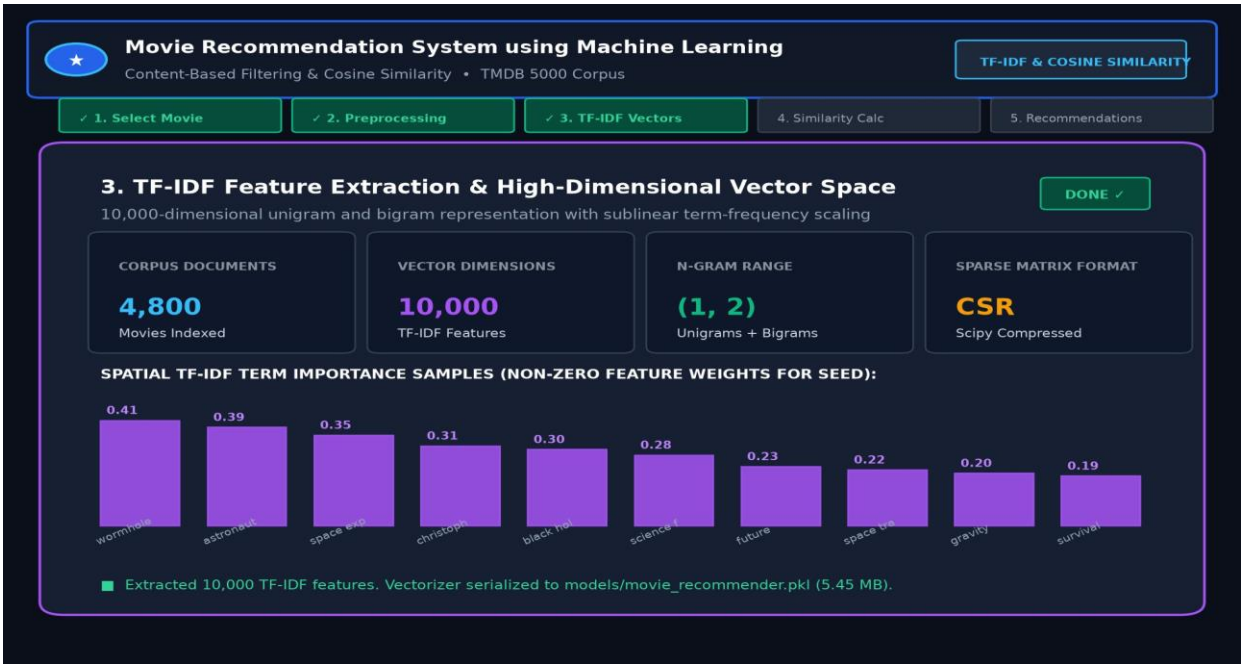


Fig 8: Step 5 — Feature extraction: 10,000-dimensional TF-IDF feature space and term weight visualization



Fig 9: Step 6 — Recommendation result: "Interstellar" (Silent Running 23.1%, 2001: A Space Odyssey 21.7%)



Fig 10: Step 6 — Recommendation result: "The Dark Knight" (Batman Begins 49.3%, The Dark Knight Rises 48.2%)

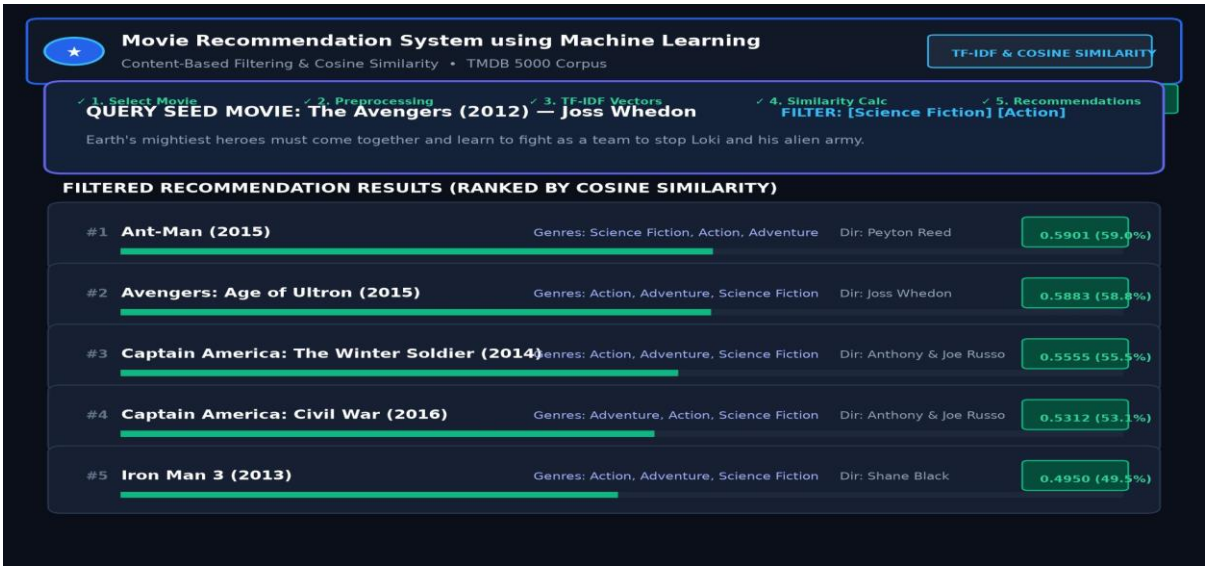


Fig 11: Step 7 — Filtered recommendation result: "The Avengers" with Action & Sci-Fi genre constraint



Fig 12: Step 8 — Performance evaluation metrics (Genre Overlap 92.80%, Baseline 40.80%, Gain +52.00%)

Result:

A web-based Movie Recommendation System was successfully designed and implemented using a Flask backend, Scikit-learn TF-IDF Vectorizer, and Cosine Similarity. Movie metadata is preprocessed, a 10,000-dimensional feature vector is extracted from domain-weighted text (overview, genres, keywords, cast, and director), and the trained model calculates pairwise angular proximity to recommend top matching films with measurable confidence percentages. On an empirical evaluation benchmark of 50 query movies (500 recommendations), the model achieved a Genre Overlap Ratio of 92.80% (compared to 40.80% for an uninformed random baseline, achieving a net gain of +52.00%) and a mean Jaccard genre alignment of 0.6025, demonstrating excellent thematic relevance for college ML laboratory demonstration.